

DAY 7 — Industry Capstone

Building and Operating a Global Network Automation Platform

Important context: Everything below is a fictional industry case study. It is designed to resemble the engineering environment and responsibilities a Lead/Staff/Principal/VP engineer might encounter in a large regulated global bank. It does **not** describe Barclays' actual internal architecture, systems, processes, or controls.

The Story: Meridian Global Bank

Our fictional organisation is **Meridian Global Bank — MGB**.

MGB operates retail banking, corporate banking, payments, markets, wealth management and internal technology services across multiple countries.

Its technology estate has grown through acquisitions, regional autonomy, technology refreshes and cloud adoption.

The result is a network environment containing approximately:

- 45,000 managed network devices
- 8 major data-centre locations
- 2 private-cloud regions
- AWS, Azure and GCP environments
- 2,000+ offices and branches
- large LAN/WLAN estate
- MPLS and Internet WAN
- SD-WAN for selected offices
- VXLAN/EVPN fabrics in newer data centres
- legacy VLAN-based data-centre networks
- several firewall platforms
- load balancers
- VPN infrastructure
- DNS/DHCP/IPAM services
- multiple network vendors
- cloud-native networking constructs
- several vendor controllers

The bank processes business-critical transactions continuously.

A network mistake can therefore affect:

Network change

|
v

Connectivity loss

|

+--> Payment service outage

|

+--> Trading platform degradation

|

+--> Branch connectivity failure

|

+--> Cloud service isolation

|

+--> Security-control failure

|

+--> Regulatory / operational risk

This is the environment into which you arrive as a **Lead Software Engineer / VP responsible for Network Automation Engineering**.

PART I — WHY THE PROJECT EXISTS

1. The Existing Business Problem

Network changes at MGB evolved organically.

Some teams use:

Engineer

|
v

SSH

|
v

Router CLI

Other teams have Python scripts:

Engineer

|
v

Python Script

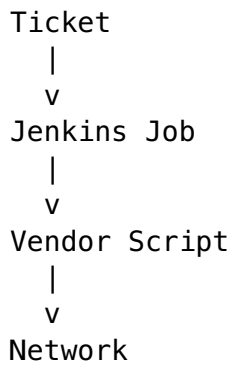
|

+--> Router 1

+--> Router 2

+--> Router 3

Another team has Jenkins:



The data-centre team uses a vendor controller.

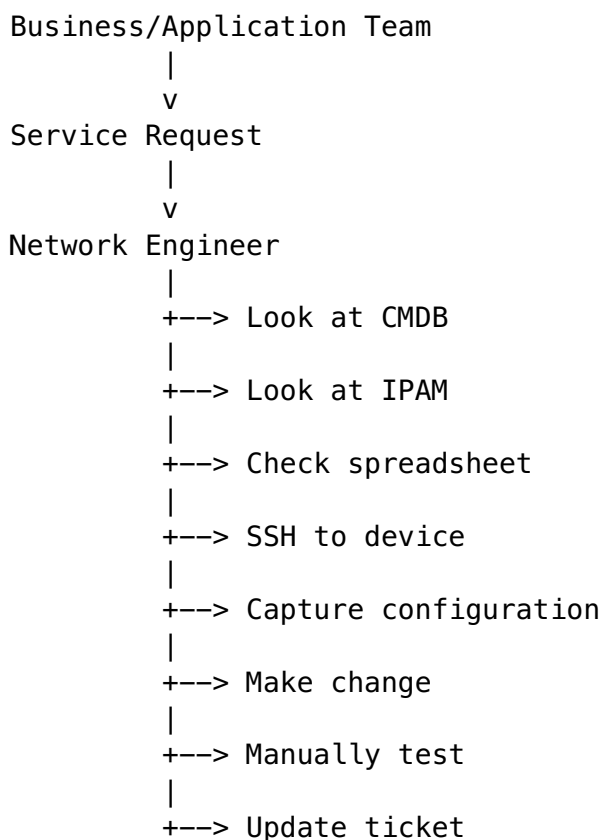
The cloud team uses Terraform.

The WAN team has its own automation framework.

The firewall team uses another vendor's management platform.

There is no common operating model.

A simple firewall or routing change might involve:



Sometimes the engineer discovers that:

CMDB says:	Router belongs to London
IPAM says:	Router belongs to UK-Core
Spreadsheet says:	Router belongs to WAN team
Device says:	Different hostname

Nobody immediately knows which system is authoritative.

Consequences

Slow delivery

A network request requiring ten minutes of actual configuration may take several days because of coordination and approvals.

Human error

An engineer pastes a correct BGP policy into the wrong router.

Another accidentally changes both members of a redundant pair simultaneously.

Configuration drift

The intended configuration and actual configuration gradually diverge.

Weak rollback

Many procedures effectively say:

| "Paste the old configuration back."

That is not reliable rollback engineering.

Poor evidence

Six months later an auditor asks:

| Who changed this route policy?

The organisation may need to correlate:

ServiceNow ticket

+

Jenkins logs

+

SSH logs

+

email

+

device logs

Limited scalability

Managing 45,000 devices using a model designed for 5,000 becomes increasingly expensive.

Business impact

These technical problems ultimately cause:

- slower application releases
- higher operational cost
- avoidable outages
- security exposure

- poor operator experience
- slower cloud expansion
- poor auditability
- difficulty standardising technology

This becomes the executive justification for the project.

2. The VP Does Not Start With Kubernetes

The first mistake would be saying:

| "We need Kubernetes, Kafka, PostgreSQL and microservices."

Those are solutions.

We first define the problems.

Business problem

Business services wait too long for network provisioning.

Network/operator problem

Network engineers spend excessive time executing repetitive changes and manually collecting evidence.

Software/platform problem

Automation is fragmented, difficult to reuse and inconsistent across vendors.

Security problem

Privileged network credentials and automation permissions are not managed consistently.

Risk/control problem

There is insufficient automated evidence that changes were:

- authorised
 - validated
 - approved
 - executed correctly
 - verified afterward
-

Organisational problem

Different network teams have built different automation stacks.

No team owns a common platform.

Constraints

The discovery phase establishes important constraints.

Constraint 1

MGB cannot replace 45,000 network devices.

The platform must work with the existing estate.

Constraint 2

Legacy CLI automation must continue temporarily.

Constraint 3

Some environments cannot accept inbound connectivity.

Automation workers must initiate connections from authorised management zones.

Constraint 4

Some devices support modern APIs.

Others support only CLI.

Constraint 5

Not every team can migrate simultaneously.

Constraint 6

The bank has strict privileged-access requirements.

Constraint 7

Important changes must still pass formal change governance.

Constraint 8

The project has limited engineering capacity.

The first year cannot solve every network problem.

Unknowns

The discovery team records unknowns rather than pretending they do not exist:

- actual device inventory accuracy
- vendor API reliability
- controller availability limits
- maximum safe device concurrency
- configuration rollback behaviour by vendor
- regional connectivity constraints
- data-sovereignty requirements
- which legacy workflows are genuinely business critical
- quality of CMDB records

A VP treats these unknowns as engineering work.

They become:

- discovery spikes
- prototypes
- experiments
- architecture risks

rather than assumptions hidden inside a design.

3. Stakeholders

The project is not simply Software Engineering versus Network Engineering.

Stakeholder	Primary concern
Network Engineering	correctness and network behaviour
Network Operations	safe operational execution
Platform Engineering	reusable software platform
SRE	reliability and recoverability
Cloud Teams	automation speed
Cybersecurity	least privilege and attack surface
Technology Risk	evidence and control effectiveness
Change Governance	appropriate approvals
Application Teams	faster provisioning
Architecture	long-term standardisation
Data/Analytics	operational insight
Senior Technology Leadership	risk, delivery and cost
Finance	investment return
Vendors	product integration/support

Conflict appears immediately.

Application teams want:

| "Why can't a subnet be provisioned in five minutes?"

Network operations says:

| "Because one bad routing change can affect thousands of services."

Security says:

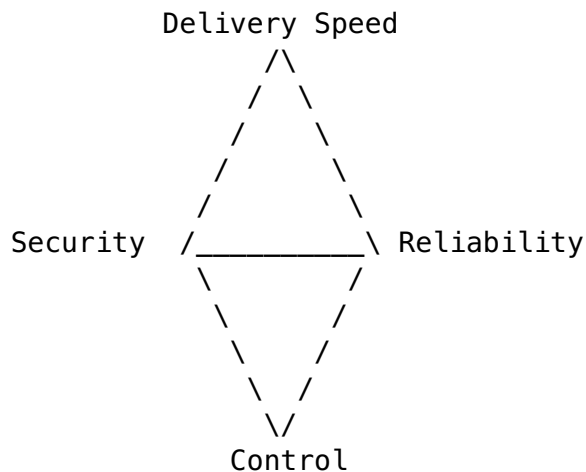
| "Automation must not become a global privileged account."

Architecture says:

| "Stop building another isolated tool."

The platform therefore cannot optimise only for speed.

The target is:



4. Product Vision

The agreed vision becomes:

Provide a secure, policy-driven, self-service network automation platform that enables authorised teams to perform validated, controlled and observable network changes consistently across the global estate.

The important word is **platform**.

The VP explicitly refuses to define success as:

"We automated 200 scripts."

Instead the organisation wants reusable capabilities.

Capability Evolution

MVP

The first release intentionally contains less functionality:

Inventory integration
Authentication / Authorization
Change API
Workflow execution
Device adapters
Pre/post checks
Audit
Configuration backup
Basic telemetry

No autonomous remediation.

No sophisticated intent engine.

No attempt to support every vendor.

Production Platform

Next:

Multi-vendor support
Policy engine
Approval workflow
Drift detection
Self-service APIs
Progressive rollout
Rich observability
Standard workflow SDK

Mature Platform

Eventually:

Desired-state networking
Policy-driven automation
Topology-aware validation
Closed-loop remediation
Analytics
Capacity prediction
Risk-based approvals

This is important VP thinking:

| **Do not confuse the final architectural vision with the first implementation milestone.**

PART II — REQUIREMENTS BECOME ARCHITECTURE

5. Functional Requirements

The main platform requirements become:

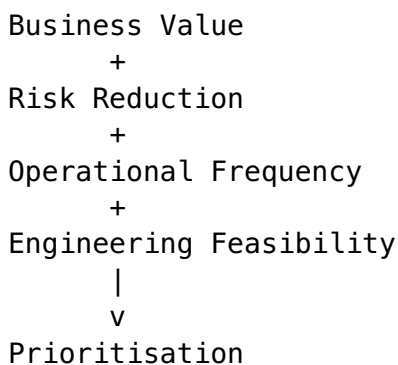
1. discover network resources
2. maintain inventory
3. integrate authoritative enterprise data
4. retrieve observed state
5. define intended changes
6. validate configurations
7. classify change risk
8. obtain approvals where required
9. execute asynchronous workflows
10. distribute work safely
11. track long-running jobs
12. perform pre-checks

- 13. progressively deploy changes
 - 14. perform post-checks
 - 15. roll back where possible
 - 16. detect configuration drift
 - 17. generate immutable audit evidence
 - 18. expose APIs
 - 19. produce operational telemetry
 - 20. integrate enterprise notifications and change systems
-

Prioritisation

Not every request receives equal priority.

MGB uses approximately:



Example:

Automating a standard branch VLAN workflow occurring 10,000 times per year is prioritised above an obscure hardware migration occurring twice.

6. Non-Functional Requirements

The following numbers are **illustrative hypothetical targets**.

Area	Initial target
API availability	99.95%
Core workflow-control availability	99.95%+
API p95 latency	<300 ms excluding device execution
Device scale	50,000 target
Concurrent tasks	bounded dynamically
Workflow-state durability	no acknowledged state silently lost
RTO	≤30 minutes for critical control plane
RPO	≤5 minutes for ordinary workflow metadata
Audit evidence	significantly stronger durability
Encryption	in transit and at rest

Area	Initial target
Authentication	enterprise identity
Privileged access	short-lived where supported
Horizontal scalability	APIs/workers/collectors
Backward compatibility	versioned public APIs
Observability	logs + metrics + traces
Multi-vendor	adapter architecture

These requirements immediately affect architecture.

For example:

Requirement: long-running workflows.

Therefore:

```

HTTP request
  |
  X Do NOT hold connection for 40 minutes

```

Instead:

```

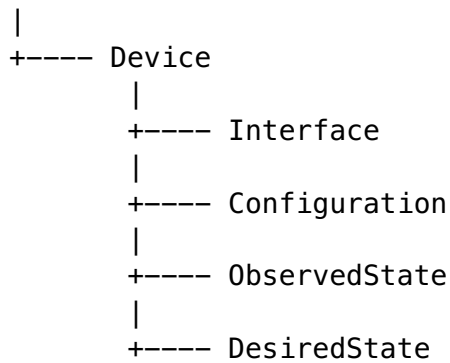
POST /changes
  |
  v
202 Accepted
Workflow ID
  |
  v
Asynchronous execution

```

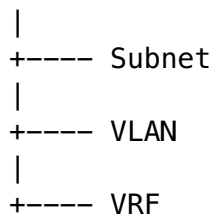
7. Domain Model

The platform team first establishes a shared language.

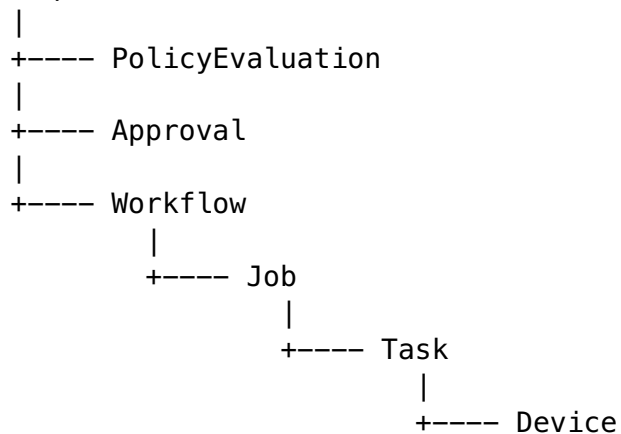
Site



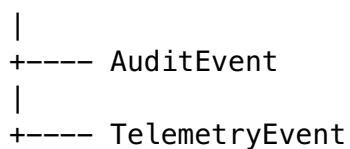
Network



ChangeRequest



Task



A particularly important distinction is:

DesiredState != ObservedState

Desired state answers:

| What should the network look like?

Observed state answers:

| What does the network appear to look like right now?

That separation later enables drift detection.

8. Source of Truth

The VP makes another important decision:

| There will not be one magical database containing all truth.

Different systems own different domains.

For example:

Information	Authority
Business service ownership	CMDB
IP allocation	IPAM
Approved config intent	Network automation platform
Device operational state	Device/controller discovery
Cloud network resources	Cloud control plane
Change approval	Enterprise change system

The platform assembles these domains.

Authoritative vs Observed

Suppose:

CMDB

Router R101 -> London

Discovery

Router R101 -> Amsterdam management network

The automation platform does **not** automatically overwrite CMDB.

It records:

Authoritative site = London

Observed site = Amsterdam

Reconciliation = CONFLICT

Depending on the field:

Conflict

|

+-> Alert owner

|

+-> Block risky change

|

+-> Create reconciliation task

Data ownership is explicit.

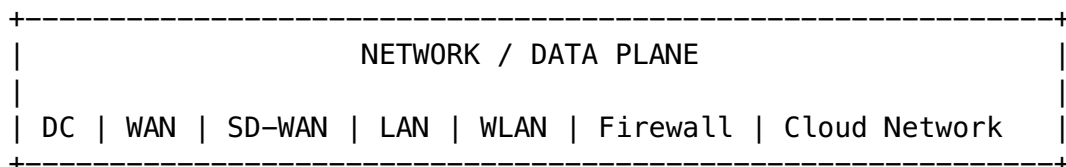
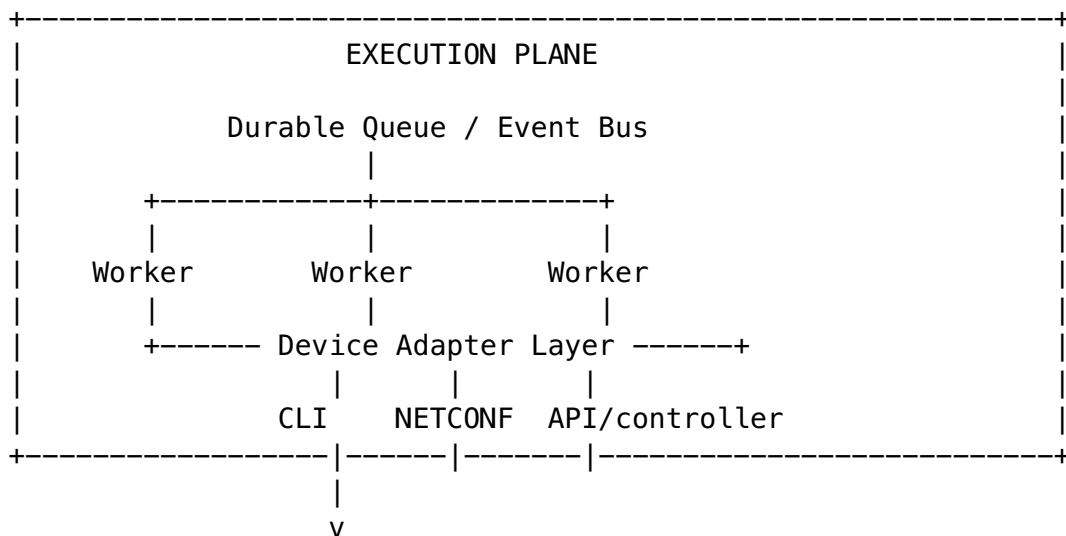
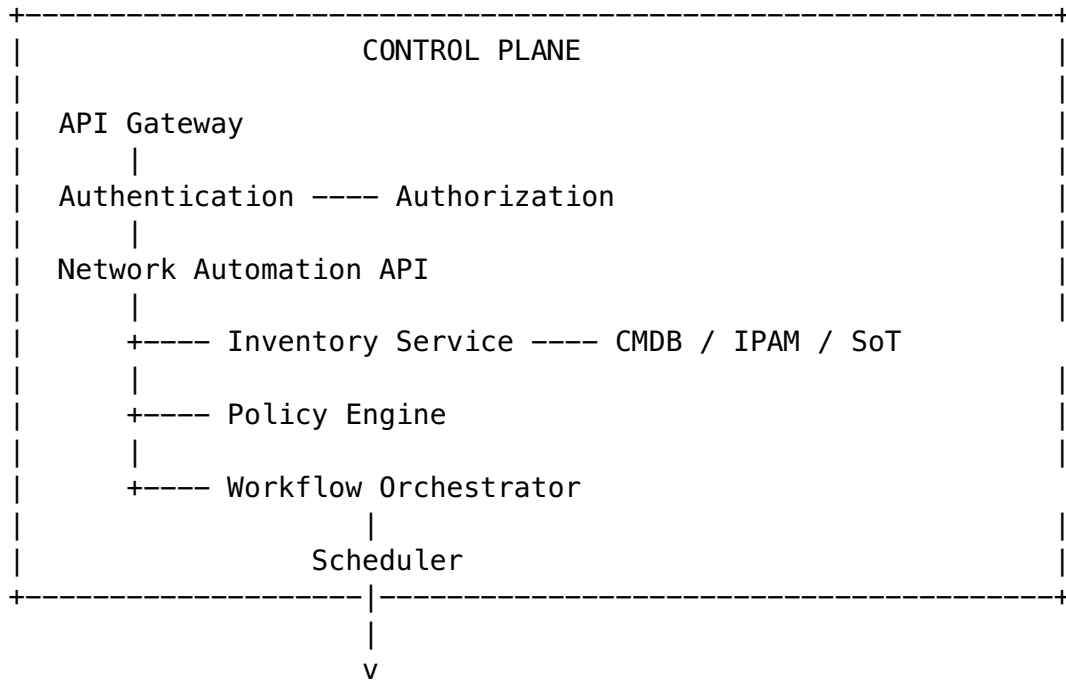
This avoids the extremely dangerous pattern:

Last writer wins = truth

9. High-Level Architecture

The first architecture proposal becomes:

USERS / ENTERPRISE SYSTEMS
 Engineers | Portal | CI/CD | ITSM
 |
 v



SUPPORTING DATA

Relational DB	-> workflow/inventory metadata
Object Storage	-> config snapshots/artifacts
Config Repository	-> intent/templates/versioning
Cache	-> frequent reads
Audit Store	-> immutable evidence

Secrets Platform -> credentials/keys/certs

OBSERVABILITY PLANE

Apps + workers + devices



Telemetry Pipeline



+-----+-----+



Logs



Metrics/Traces



+-----+-----+



Dashboards / Alerts / Analytics

Architectural Decision 1 — Asynchronous Workflows

Problem

Network operations can take minutes or hours.

Options

1. synchronous API calls
2. background threads
3. durable asynchronous workflows

Selected

Durable asynchronous workflows.

Why

Execution must survive:

- API restart
- worker restart
- temporary device failure
- approval delay
- maintenance window
- retry

Trade-off

Much more software complexity.

Failure implication

Workflow state becomes a critical business record.

PART III — THE CENTRAL CHANGE STORY

10. Routing Policy Update Across 2,000 Devices

A corporate network team needs to introduce a new routing policy.

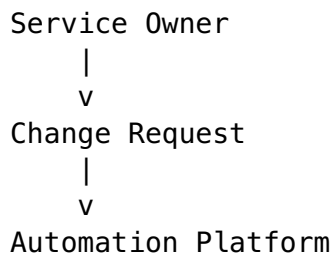
The policy must update approximately:

- 900 WAN edge routers
- 600 data-centre border devices
- 500 cloud/connectivity routers

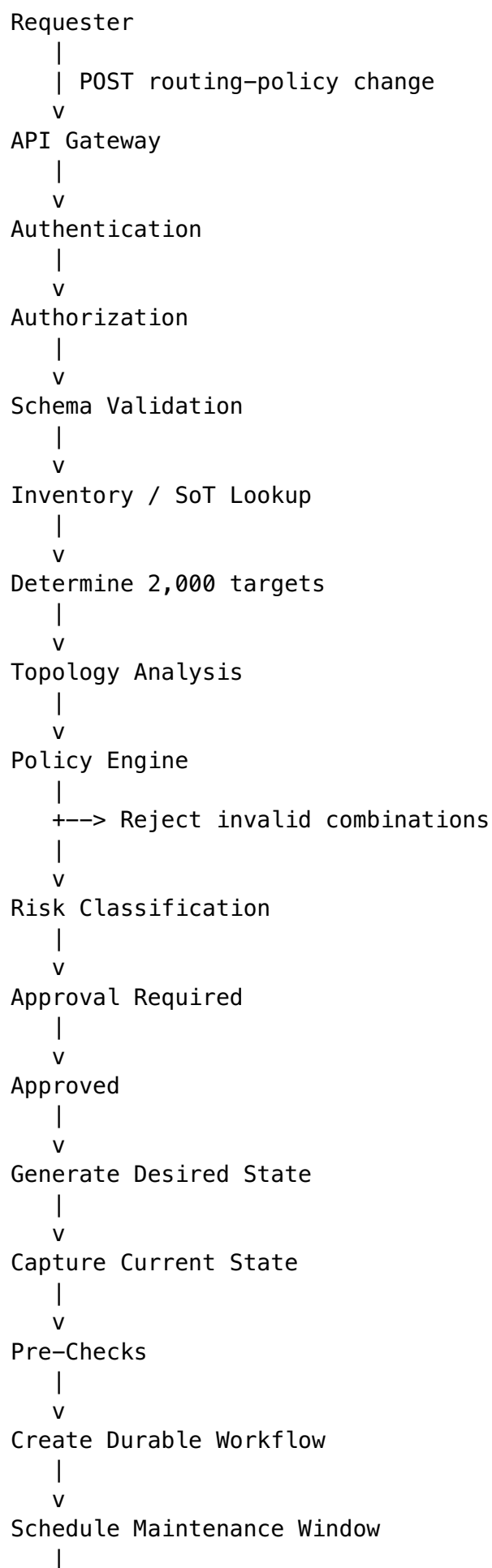
across several regions.

Without automation, this could require many engineers and several maintenance windows.

With MGB's platform:



The complete sequence becomes:





The important architectural idea is that the request does not immediately become:

```

for device in devices:
    push_config(device)

```

There are many safety layers before device modification.

11. Device Integration

MGB has heterogeneous devices.

Some support:

SSH/CLI

Widely available but less structured.

NETCONF

Structured configuration using YANG models.

RESTCONF

HTTP-based structured configuration.

REST APIs

Common for controllers and appliances.

gNMI

Useful for structured configuration/telemetry in modern environments.

SNMP

Primarily useful for monitoring and limited legacy operations rather than complex configuration.

Vendor controllers

Useful where the controller is already the authoritative management interface.

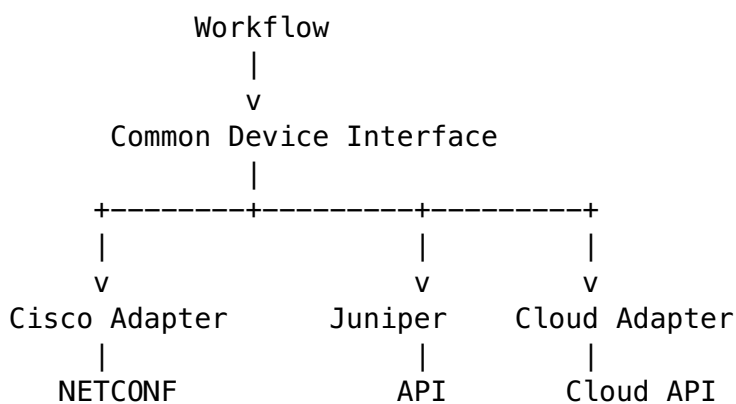
Adapter Architecture

The workflow engine never contains:

```
if vendor == Cisco ...  
else if vendor == Juniper ...  
else if vendor == ...
```

throughout the codebase.

Instead:



The abstraction might conceptually support:

```
get_state()
validate_change()
apply_change()
verify_change()
rollback()
capabilities()
```

No implementation code is required to understand the design.

Why Abstraction Matters

Without it:

```
Workflow A -> Vendor logic
Workflow B -> Vendor logic
Workflow C -> Vendor logic
```

Every vendor upgrade touches many workflows.

With adapters:

```
Workflows
  |
Standard capability model
  |
Adapter
```

But abstraction is not magic.

A Cisco router running one software generation may behave differently from another.

Therefore capability detection matters:

```
Device
|
+--> Vendor
+--> Model
+--> OS version
+--> Protocol support
+--> Feature capabilities
```

This detail will later cause our production incident.

12. Model-Driven Networking

Where supported, MGB prefers structured models.

```
YANG
|
+--> Data model
    |
    +--> NETCONF
    +--> RESTCONF
    +--> gNMI / structured telemetry
```

OpenConfig can provide vendor-neutral models for many common capabilities.

Benefits include:

Validation

The platform can validate structured values before sending them.

Portability

Intent becomes less tightly bound to command syntax.

Diffing

Structured trees are easier to compare than raw text.

Telemetry alignment

Configuration and operational state can share common models.

But MGB does **not** declare:

```
| "CLI automation is banned."
```

Some critical legacy devices simply do not support modern structured interfaces.

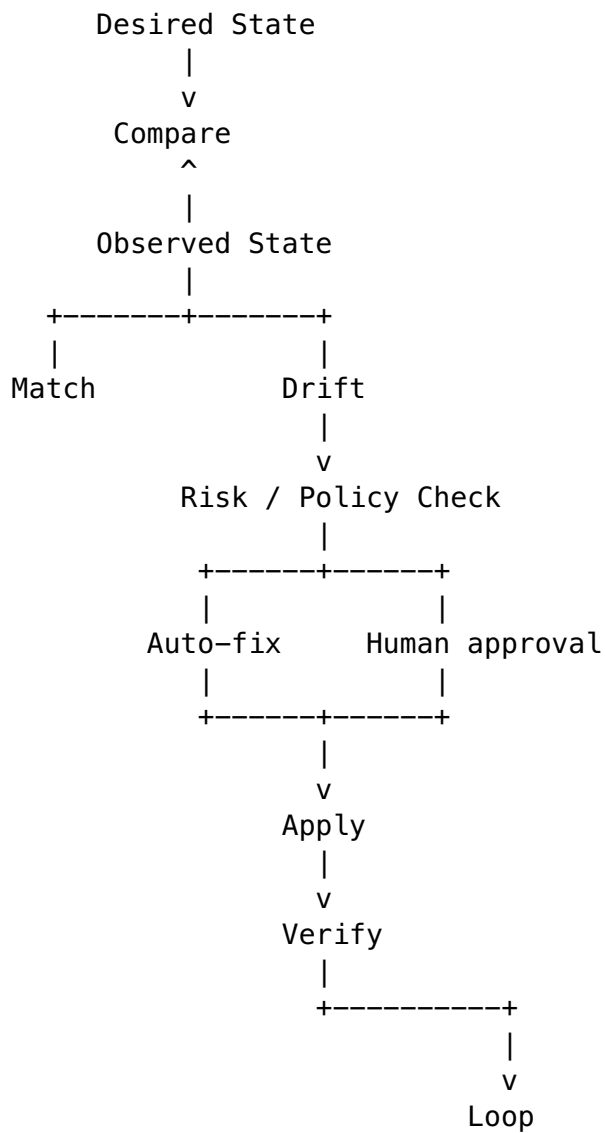
So the real architecture is:

```
Modern structured automation
      +
Legacy compatibility
```

not technological purity.

13. Desired-State Architecture

Over time the platform evolves from command automation toward reconciliation.



This resembles Kubernetes:

```

graph TD
    DS[Desired State] --> C[Controller]
    C --> O[Observe]
    O --> R[Reconcile]
  
```

However, networking introduces substantial blast-radius risk.

Automatically restoring:

```

graph TD
    NTP[NTP server]
    ST[syslog target]
    ADM[approved device metadata]
  
```

may be acceptable.

Automatically modifying:

core BGP policy
firewall rule
WAN routing
EVPN control plane

without human review may be unacceptable.

Automation level depends on risk.

PART IV — DISTRIBUTED EXECUTION

14. Workflow State Machine

The routing change is represented as:

```
Pending
|
Validating
|
Awaiting Approval
|
Scheduled
|
Running
|
Verifying
|
Succeeded
```

Failure path:

```
Running
|
Failed
|
+---> Retry
|
+---> Rolling Back
|
+---> Rolled Back
|
+---> Manual Intervention
```

Every workflow receives a durable identifier:

Workflow ID:
WF-2026-814391

Each device operation becomes a task:

WF-2026-814391

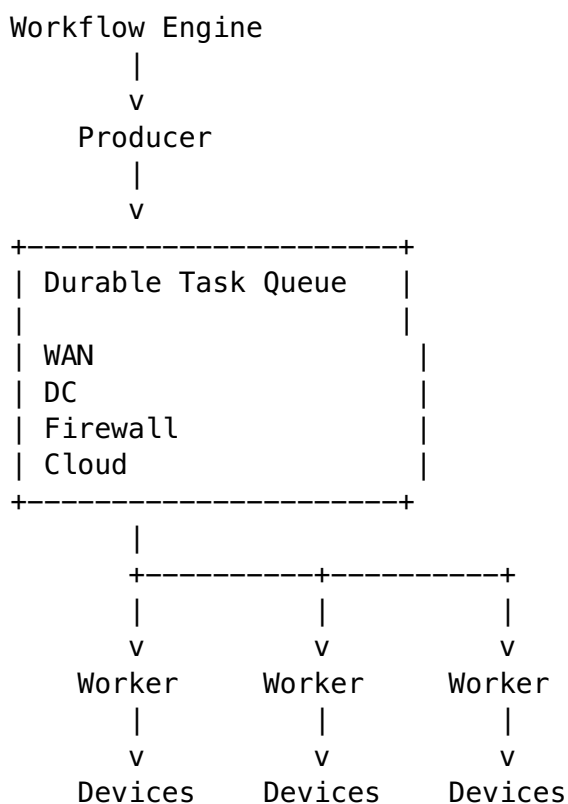
```
|  
+-- T001 -> Router-LON-001  
+-- T002 -> Router-LON-002  
+-- T003 -> Router-FRA-007  
...
```

If a worker crashes halfway through, the workflow does not disappear.

Another worker can recover state.

15. Queue and Worker Architecture

The execution plane looks like:



Workers scale horizontally.

But the platform does **not** equate scalability with:

| "Open 20,000 SSH sessions."

That could overload:

- devices
- controllers
- WAN links
- TACACS/RADIUS systems
- secrets platform
- monitoring platforms

So concurrency is bounded.

For example:

Global limit

```
|  
+--> Region limit  
|  
+--> Site limit  
|  
+--> Vendor/controller limit  
|  
+--> Device-family limit
```

This is backpressure.

The system intentionally slows itself to protect the infrastructure it manages.

Scaling

For 100 devices:

1 queue
small worker pool

At 50,000:

Partitioned queues
Regional worker pools
Capability-aware scheduling
Priority classes
Per-domain limits
Horizontal autoscaling

But again:

| Worker count should not determine network concurrency.

A worker may be available while network policy says:

maximum 10 concurrent changes in this site

16. Delivery Semantics and Idempotency

The platform assumes task delivery may be:

| **at least once**

Therefore:

Task T100

|
Worker A

|
Worker dies before ACK

|
Queue redelivers

|
Worker B

Worker B may receive a task that Worker A already executed.

This requires idempotency.

Each operation has an operation ID:

operation_id = 0P-991827

The system asks:

Has 0P-991827 already completed?

or better:

Is desired state already present?

Example:

Bad retry:

Add route-policy statement
Add route-policy statement

Possible duplicate configuration.

Safer approach:

Desired policy version = 17

Current policy version = 17

=> Nothing to do

Exactly-once execution is extremely difficult across:

queue
+
worker
+
network
+
remote device

So mature designs combine:

- at-least-once delivery
 - idempotent operations
 - deduplication
 - observed-state verification
-

17. Conflicting Changes

Two workflows target Router R101.

```
Workflow A -----\
                                   > Router R101
Workflow B -----/
```

A dangerous result could be:

A reads version 10

B reads version 10

A writes version 11

B writes version 11 based on stale state

Potential strategies include:

Device lock

Simple but can reduce concurrency.

Resource lock

Lock only BGP policy rather than entire router.

More concurrency, more complexity.

Lease

A worker owns a temporary lock that expires if it dies.

Optimistic concurrency

Expected config version = 10

Actual config version = 11

=> reject operation

MGB chooses a combination:

- resource/device leases for high-risk writes
- state-version checks
- conflict detection

Read operations remain concurrent.

18. Partial Failure

A later rollout targets 1,000 devices.

Result:

850 succeeded
100 unreachable
30 configuration rejected
20 failed post-validation

There is no sensible global ACID transaction.

You cannot perform:

BEGIN TRANSACTION;

update 1000 routers;

ROLLBACK;

Network devices are independent distributed systems.

The workflow therefore reports:

Workflow: PARTIALLY FAILED

Succeeded:	850
Unreachable:	100
Config rejected:	30
Post-check failure:	20

Decision logic might be:

Unreachable devices

Retry with bounded exponential backoff.

Configuration rejection

Do not retry blindly.

Investigate capability/version mismatch.

Post-check failure

Potentially rollback affected devices.

Systemic failure threshold

Pause the entire rollout.

This distinction matters:

1 unreachable device

is different from:

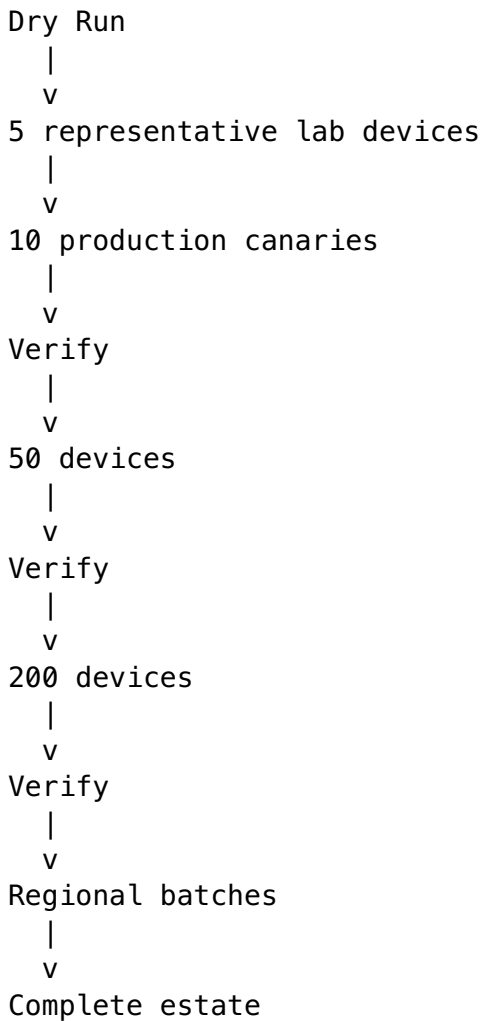
20% of devices suddenly losing BGP routes

The workflow engine therefore understands failure categories.

PART V — SAFE CHANGE ENGINEERING

19. Progressive Rollout

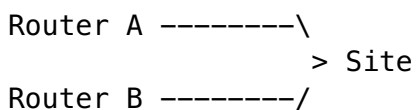
For the 2,000-device routing policy:



But simple percentages are not enough.

The platform considers topology.

For redundant routers:



Never update both simultaneously.

Therefore:

Batch 1 -> A-side devices
Verify redundancy
Batch 2 -> B-side devices

Failure thresholds might include:

>2% post-check failure -> pause

BGP route count deviation above threshold -> pause

Critical application health decline -> pause

Hypothetical thresholds must be calibrated using operational evidence.

20. Pre-Change Validation

Before applying the routing policy, the platform checks:

Device reachable?
|
Current state retrievable?
|
Expected config version?
|
Correct software version?
|
Redundant peer healthy?
|
Routing neighbors healthy?
|
Capacity available?
|
Maintenance window active?
|
Policy permitted?
|
Dependency healthy?
|
PASS

This highlights an important concept.

A configuration can be:

syntactically valid

and still:

operationally catastrophic

Example:

deny 0.0.0.0/0

may be completely valid syntax.

It may also remove the routes required for connectivity.

21. Post-Change Verification

After execution the platform validates several layers.

Configuration

Did the desired configuration appear?

Control plane

Are BGP/OSPF relationships healthy?

Routing state

Are expected routes still present?

Network reachability

Can critical probes pass?

Application signals

Did transaction failures increase?

Device health

CPU, memory, errors, interface state.

So:

```
Config Applied
  |
  X
```

does not automatically mean:

```
Change Successful
```

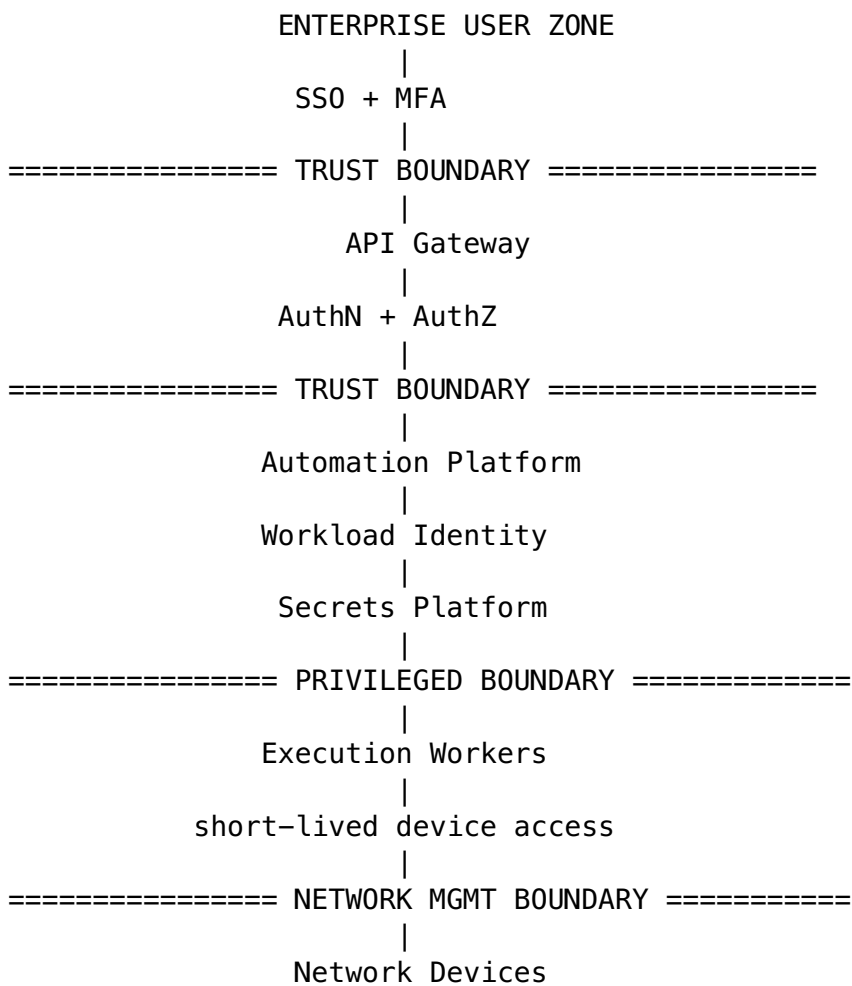
Instead:

```
Config Applied
  |
  v
Operational State Correct?
  |
  v
Service Health Correct?
  |
  v
SUCCESS
```

PART VI — SECURITY, RISK AND CONTROLS

22. Security Architecture

MGB models several trust boundaries.



Human identity

Users authenticate using central SSO.

Conceptually:

OIDC
+
MFA
+
enterprise identity

Machine identity

Services use:

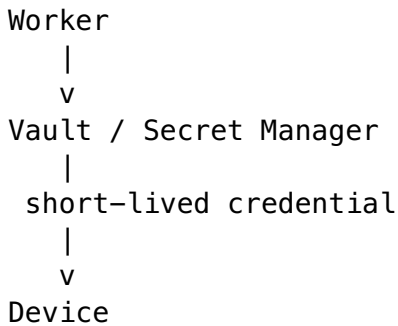
- workload identity
- mTLS

- service accounts where unavoidable
- short-lived credentials

Device access

Long-lived passwords embedded in scripts are eliminated.

Instead:



Where devices cannot support dynamic credentials, access is still centrally stored and aggressively rotated.

Platform protection

Additional controls include:

- network segmentation
 - API input validation
 - encryption at rest
 - encryption in transit
 - secrets redaction
 - workload isolation
 - restricted administrative interfaces
 - signed artifacts
 - dependency scanning
-

23. Authorization

Roles include:

Role	Capability
Read-only operator	inspect state
Change requester	propose change
Network engineer	create technical change
Approver	approve authorised scopes
Platform administrator	platform administration
Security auditor	read evidence

RBAC alone eventually becomes insufficient.

Suppose a network engineer has:

```
role = network-engineer
```

That should not necessarily permit modification of every global router.

Contextual policy adds:

```
User.role = NetworkEngineer
```

AND

```
User.region = EMEA
```

AND

```
Device.region = EMEA
```

AND

```
Change.type permitted
```

AND

```
Maintenance window valid
```

This resembles ABAC/policy-based authorization.

24. Major Engineering Risks

Risk	Consequence	Mitigation	Residual risk
Global misconfiguration	broad outage	policy, canary, limits	unknown semantic failure
Credential compromise	privileged network access	vault, rotation, mTLS	compromised authorised workload
Wrong SoT data	wrong device targeted	reconciliation, validation	stale upstream data
Automation defect	repeated incorrect change	tests, progressive rollout	untested scenario
Queue overload	delayed operations	backpressure	prolonged incident surge
Vendor incompatibility	failed changes	capability model, contract tests	undocumented vendor behaviour
Network partition	partial execution	durable state, verification	ambiguous completion
Rollback failure	extended outage	tested compensation	state may have changed
Insider misuse	malicious change	SoD, audit, least privilege	authorised privileged misuse
Platform outage	automation unavailable	HA/DR	manual operations may be needed
DB corruption	lost state	replication/backups	recovery lag

Risk	Consequence	Mitigation	Residual risk
Vendor controller outage	blocked workflows	dependency isolation	vendor SLA

Risk can never be reduced to zero.

A VP asks:

| Is the residual risk understood and acceptable relative to the business value?

25. Controls as Architecture

Controls are not simply paperwork placed around software.

They are implemented within the platform.

Preventive

RBAC
Policy validation
Schema validation
Change approval
Concurrency control
Canary rollout
Maintenance windows

Detective

Drift detection
Post-checks
Audit monitoring
Security alerts
Telemetry

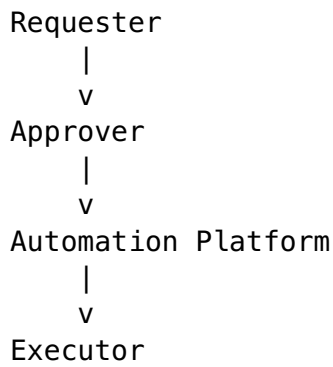
Corrective

Rollback
Restore
Credential rotation
Failover
Incident runbooks

The architecture itself becomes part of the control environment.

26. Segregation of Duties

For sufficiently high-risk changes:



The requester should not silently approve their own high-risk change.

The audit system records:

Change ID
Requester
Business reason
Affected resources
Previous state
Desired state
Policy evaluation
Risk score
Approver
Approval timestamp
Execution workflow
Exact actions
Device responses
Post-check results
Rollback actions
Final state

Six months later:

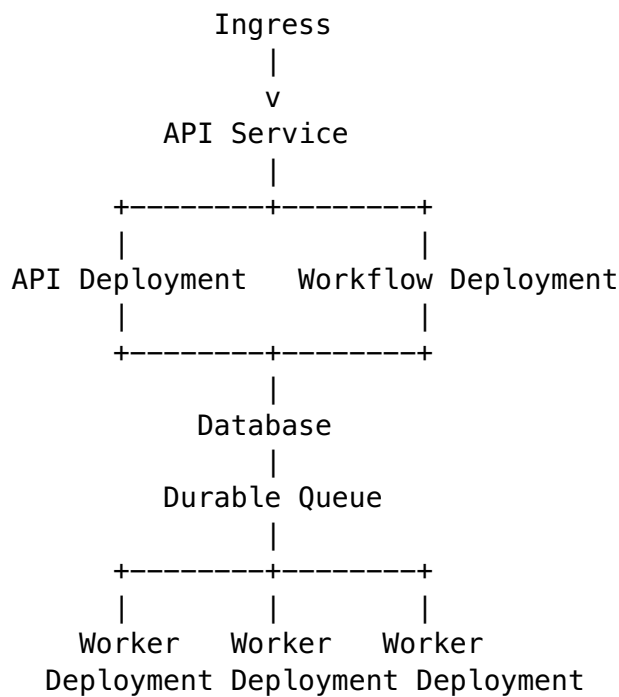
| Who changed Router R101?

is answerable without forensic archaeology.

PART VII — RUNNING THE PLATFORM

27. Kubernetes Deployment

The platform runs primarily on the bank's standard Kubernetes platform.



Scheduled Discovery
|
CronJob

Node-level telemetry where needed
|
DaemonSet

Mapping:

API	-> Deployment
Orchestrator	-> Deployment
Workers	-> Deployment
Discovery	-> CronJob
Telemetry agent	-> DaemonSet where appropriate

Stateful enterprise services are preferably consumed as approved managed/platform services where available rather than casually operating databases inside application clusters.

Kubernetes controls

The deployment uses:

- readiness probes
- liveness where appropriate
- PodDisruptionBudgets
- resource requests
- resource limits
- HPA

- anti-affinity
- multi-AZ placement
- NetworkPolicy
- secure CNI configuration
- secret integration

A critical lesson:

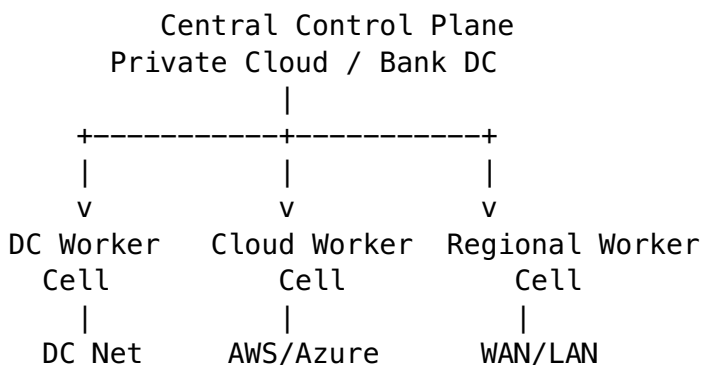
Kubernetes provides infrastructure primitives. It does not automatically make the application resilient.

28. Hybrid Hosting Decision

The network exists everywhere.

Therefore a single hosting location creates problems.

MGB selects a **hybrid control/execution topology**.



The central platform handles:

- policy
- workflows
- inventory
- audit
- control

Execution cells are positioned where connectivity and policy require them.

Why not simply place everything in public cloud?

Because:

- private-device reachability may depend on DC connectivity
- network partitions could isolate automation
- latency may matter
- security policies may limit management access
- data location may matter
- dedicated cloud connectivity itself becomes a dependency

Likewise, keeping everything on premises makes cloud automation unnecessarily dependent on on-prem connectivity.

Hybrid is therefore deliberate.

29. Data Architecture

MGB deliberately avoids one giant database.

Relational Database

- |
- +++> inventory metadata
- +++> workflows
- +++> jobs
- +++> approvals
- +++> policy results

Cache

- |
- +++> frequently read metadata
- +++> short-lived session/reference data

Object Storage

- |
- +++> configuration snapshots
- +++> large workflow artifacts
- +++> diagnostic bundles

Search Platform

- |
- +++> searchable operational logs

Audit Store

- |
- +++> durable control evidence

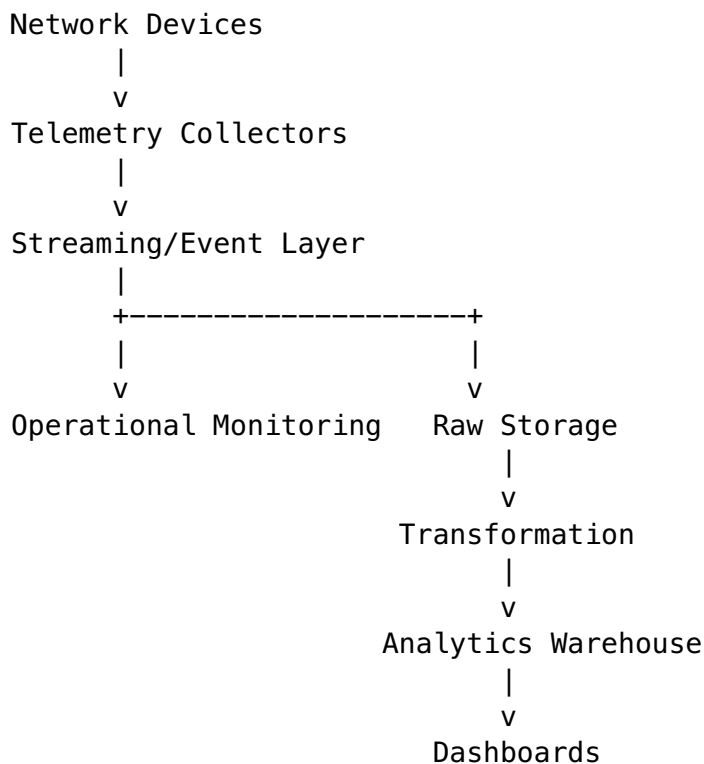
Analytics Platform

- |
- +++> long-term trends
- +++> success/failure analysis
- +++> risk reporting

Different workloads require different access patterns.

30. Telemetry Pipeline

The network itself produces substantial telemetry.



Possible analytical platforms could include technologies such as Snowflake, BigQuery or Redshift depending on enterprise standards.

Streaming versus batch

Streaming supports:

route changes
device failures
BGP events
automation verification

Batch supports:

daily compliance analysis
capacity reports
historical trends

Telemetry engineering also addresses:

- event duplication
 - ordering
 - schema changes
 - missing records
 - retention
 - late arrival
 - data quality
-

31. Observability

Platform observability spans:

Logs

Structured event details.

Metrics

Aggregated numerical behaviour.

Traces

Request journey across distributed components.

Correlation is designed from the beginning:

```
Request ID
  |
  v
Workflow ID
  |
  v
Task ID
  |
  v
Device ID
  |
  v
Trace ID
```

Example:

```
REQ-801
  |
WF-223
  |
TASK-882
  |
RTR-LON-19
```

An operator can move from API request to the exact device task.

Important metrics

API request rate
API p95 latency
Workflow duration
Queue depth
Oldest queue message
Worker utilisation
Device connection failure
Change success
Post-check failure
Rollback rate
Drift count
Adapter error rate

32. SLI/SLO Design

The bank distinguishes:

Platform Health

from:

Managed Device Health

For example:

API availability SLI

Successful API responses / eligible API requests.

Hypothetical SLO:

99.95%

Workflow-control reliability

Percentage of accepted workflows whose orchestration state is preserved correctly.

Job-processing latency

Time between ready-to-run and worker start.

Telemetry ingestion

Percentage delivered within required latency.

Automation result

Change outcome.

But the platform cannot promise:

| 99.99% of routers will always accept changes.

The router may be:

- offline
- overloaded
- unreachable
- unsupported
- broken

So reports clearly separate:

Platform failure

vs

Managed-infrastructure failure

33. Alerting

MGB does not alert on every internal exception.

Good alert:

| High-risk workflow post-validation failure above threshold in EMEA.

Bad alert:

| Worker 17 returned exception.

A useful alert should generally be:

User/Service Impact

+

Actionable

+

Meaningful Severity

Examples:

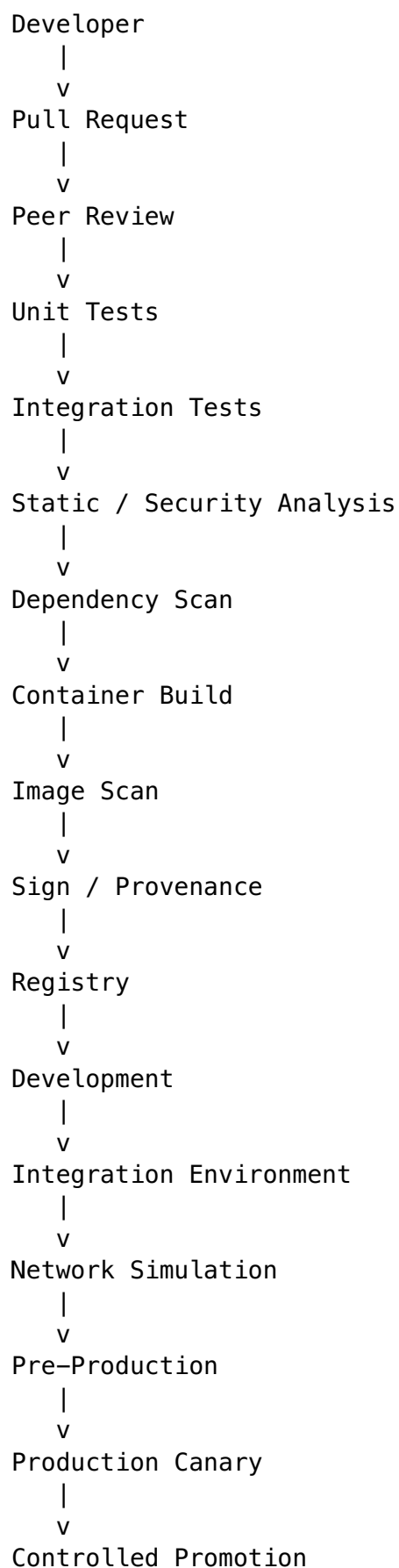
- workflow execution queue has exceeded safe delay
- multiple regions cannot access secrets service
- elevated device write failures
- post-change health failures exceed threshold
- workflow state persistence unavailable
- audit pipeline losing records

Events are deduplicated to prevent hundreds of identical alerts during one common failure.

PART VIII — SOFTWARE DELIVERY

34. CI/CD

A platform code change flows through:



Importantly:

| Application deployment and network-change deployment are separate risks.

The automation platform itself is progressively deployed.

The workflows executed by that platform are also progressively deployed.

35. Testing

Different test layers prove different things.

Test	What it proves
Unit	isolated logic works
Integration	services/stores integrate
Contract	adapter/API expectations remain compatible
Workflow	state transitions work
Adapter	vendor implementations behave correctly
Network simulation	configuration affects simulated network correctly
E2E	complete platform path
Failure	recovery logic works
Performance	scale limits understood
Security	attack surface tested
DR	recovery process works

For network automation, adapter testing becomes particularly important.

Test matrices include:

Vendor
x
Model
x
OS version
x
Feature

This becomes essential later.

PART IX — GETTING INTO PRODUCTION

36. Initial Production Release

MGB does not announce:

| "Next Monday all 45,000 devices will be automated."

The progression is:

Developer Simulation
|
Lab
|
Non-production environment
|
10 low-risk production devices
|
50 devices
|
One office domain
|
One regional service
|
Several regions
|
Global adoption

Progression depends on evidence.

For example:

Success rate > threshold
No Severity-1 incident
Rollback tested
Audit evidence accepted
Operational runbook complete
Support team trained
Capacity headroom demonstrated

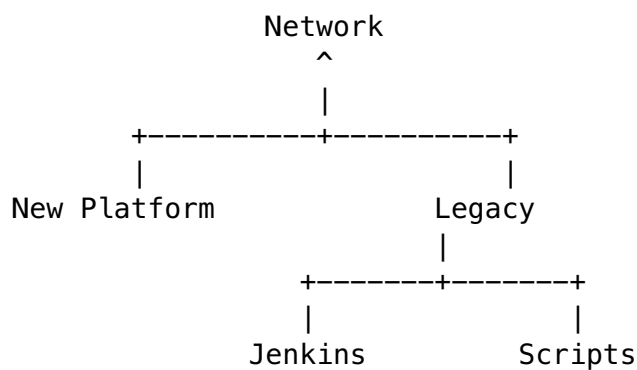
The exit criterion is not:

| "The project plan says phase two starts today."

37. Legacy Migration

Existing automation does not disappear.

Initially:



Migration follows:

Discover
|
Classify workflows
|
Standardise high-value patterns
|
Wrap selected legacy automation
|
Build native replacement
|
Move consumers
|
Measure usage
|
Deprecate
|
Retire

A temporary adapter may allow:

New Platform API
|
v
Legacy Jenkins Job

This is not elegant.

It is intentionally transitional.

That is **evolutionary architecture**.

PART X — THE PRODUCTION INCIDENT

38. Routing Policy Incident

Nine months after the first production release, MGB has grown confident in its routing automation.

The team releases routing policy version 24.

The change has passed:

- code review
- unit tests
- integration tests
- policy validation
- lab testing
- production canary

The rollout targets 1,400 WAN routers.

Initial canaries succeed.

The workflow progresses to a regional batch.

Then monitoring detects a sudden drop in advertised routes.

What happened

The platform's common routing-policy model translated the intended rule into vendor-specific configuration.

For newer versions of one router OS:

policy behaviour = expected

But an older OS release interpreted one generated policy sequence differently.

On those devices:

Expected:

Allow required regional routes

Actual:

Required routes excluded

Several routers therefore withdrew legitimate routes.

Blast Radius

The progressive rollout had already limited the impact.

Approximately 14 branch aggregation routers in one region were affected before automation paused.

Some applications experienced intermittent connectivity.

The rest of the 1,400-device rollout never happened.

Without progressive rollout, the incident might have become global.

Detection

The configuration post-check passed because:

Desired configuration existed.

BGP sessions also remained:

UP

But a telemetry guard detected:

Expected route count
|
v
Sudden significant decline

and application-path probes began failing.
This triggered the automation safety threshold.

Response

Route Anomaly Detected
|
v
High-Severity Alert
|
v
Incident Declared
|
v
Workflow PAUSED
|
v
No New Devices Changed
|
v
Affected Devices Identified
|
v
Rollback Started
|
v
Route State Recovered
|
v
Application Validation
|
v
Incident Stabilised

Technical Root Cause

The adapter generated logically incorrect configuration for a particular OS version.
The platform's abstraction assumed two software generations were semantically equivalent.
They were not.

Process Contributor

Canary selection covered:

- region
- device role
- network topology

but did **not** adequately cover:

- OS version diversity

All canaries happened to run newer software.

Control Contributor

Post-change validation checked:

BGP session up?
configuration applied?

It did not enforce:

Expected critical route set still present?

for this change class.

Why Protection Failed

Several controls existed.

None individually represented the full safety property.

Valid syntax	YES
Adapter succeeded	YES
BGP session alive	YES
Config applied	YES

Network behaviour safe NO

This is realistic engineering:

Systems often fail in the gap between what we checked and what we actually needed to know.

39. Post-Incident Engineering

The post-incident review results in permanent changes.

1. Capability matrix improved

Adapter capability now includes:

vendor
model
OS family
OS version
feature behaviour

2. Canary selection improved

Canary selection becomes multidimensional:

Region
+
Role
+
Vendor
+
Model
+
OS version
+
Topology

3. Route semantic validation added

Before rollout:

Policy
|
Simulated evaluation
|
Expected route-set comparison

4. Better post-checks

Critical routing changes now validate:

- expected prefixes
 - unexpected withdrawals
 - peer state
 - next-hop behaviour
 - application reachability
-

5. Failure threshold tightened

Route withdrawal anomalies trigger an immediate automated pause.

6. Test environment expands

Older supported network OS images are added to the compatibility lab.

The VP Lesson

The lesson is not:

| "Someone forgot a test."

The deeper lesson is that safety required alignment across:

Abstraction design
+
Compatibility management
+
Testing
+
Rollout
+
Observability
+
Risk classification

An architecture decision created a long-term operational obligation.

PART XI — DISASTER RECOVERY

40. Primary Platform Region Failure

Suppose the primary private-cloud region fails.

The platform design has a secondary recovery region.

Critical data is replicated.

Primary Region
X

Replicated state
|
v

Secondary Region

But network automation cannot simply say:

restart all workflows

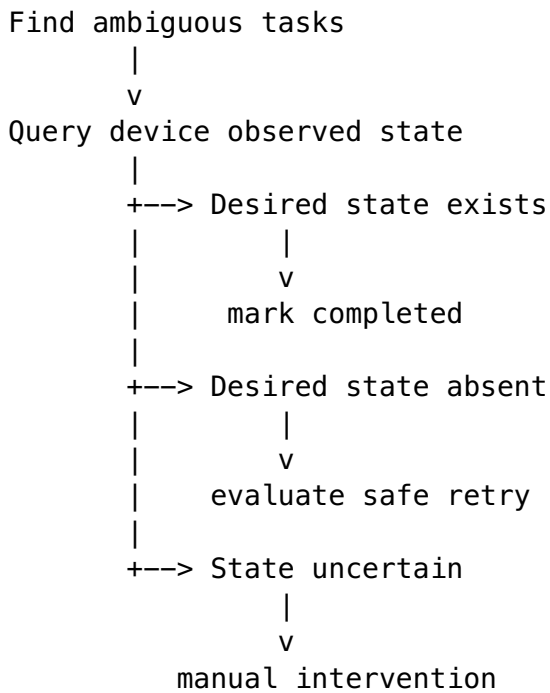
Suppose a worker sent a command before the outage but never recorded success.

After failover:

Task state = UNKNOWN

Blind retry may repeat a network change.

Therefore recovery follows:



Illustrative targets might be:

Critical control-plane RT0 <= 30 minutes
Workflow metadata RPO <= 5 minutes
Audit evidence closer to zero-loss design

Execution may intentionally remain paused after DR until network state is reconciled.

Availability is not the same as reckless continuation.

PART XII — SCALE

41. 10,000 → 50,000 Devices

At 10,000 devices, one discovery cycle may be manageable.

At 50,000:

50,000 devices
x
multiple telemetry streams
x
configuration snapshots
x
workflow events

changes architecture substantially.

API layer

Usually horizontally scalable.

Workers

Horizontally scalable but bounded by device/network limits.

Queue

Requires partitioning and capacity management.

Inventory database

Potential bottleneck.

Needs:

- indexing
 - partitioning where justified
 - read optimisation
 - archival
-

Discovery

Must be staggered.

Bad:

00:00 -> discover all 50,000 devices

Better:

distributed schedules
+
event-driven refresh
+
incremental discovery

Telemetry

Often the greatest scale challenge.

Configuration changes may produce thousands of events per day.

Telemetry may produce millions or billions.

It therefore gets an independent architecture.

42. Performance Problems

Problem: Inventory API becomes slow

Investigation shows expensive joins and unbounded responses.

Fixes:

indexes
pagination
query redesign
cache
precomputed views

Trade-off:

Caching creates freshness concerns.

Problem: Queue backlog

Possible causes:

- worker shortage
- device connectivity degradation
- overly conservative concurrency
- dependency outage

Possible fixes:

- more workers
- partitioning
- prioritisation
- batching

But adding workers is useless if the limiting factor is:

vendor controller allows 20 sessions

Problem: Database growth

Audit/workflow records grow rapidly.

Possible response:

Hot operational DB
|
archive
|
Long-term immutable storage

Problem: Telemetry overload

Use:

- sampling where appropriate
- aggregation
- tiered retention
- compressed formats
- partitioned streaming

- lifecycle policies

But security/audit evidence should not be casually sampled away.

PART XIII — ECONOMICS

43. Total Cost of Ownership

The VP evaluates:

Infrastructure
+
Software licences
+
Data storage
+
Network connectivity
+
Engineering
+
On-call
+
Support
+
Vendor support
+
Security compliance
+
Migration
+
Training

Suppose an open-source database costs no licence fee but requires four specialist engineers to run reliably.

A managed database might have a higher cloud bill but lower operating cost.

Therefore:

Cheapest Component
!=
Lowest TC0

Likewise, retaining years of high-cardinality telemetry may technically be possible but economically unjustified.

Retention becomes tiered according to value.

44. Build vs Buy

MGB deliberately mixes approaches.

Build

Bank-specific network workflows

They encode institutional knowledge and topology.

Policy/integration layer

Strongly tied to MGB's governance, identity and network model.

Domain APIs

Strategic interface for internal consumers.

Potentially Buy / Managed

Database

If an approved resilient managed service exists.

Secrets management

No reason for the application team to invent a vault.

Observability

Usually better built on enterprise standards.

Workflow technology

A durable orchestration engine may be bought/open-source rather than built from scratch.

Network controllers

Vendor controllers may remain appropriate for vendor-specific capabilities.

Build/Buy Decision Framework

Strategic differentiation?

Operational expertise?

Time to market?

Security constraints?

Cost?

Support?

Lock-in?

Exit strategy?

A VP does not adopt:

build everything

or:

buy everything

as ideology.

PART XIV — ORGANISATION

45. Ownership Model

MGB defines ownership clearly.

Network Automation Platform Team

Owns:

- APIs
- workflow engine
- platform runtime
- SDK
- adapter framework
- reliability

Network Domain Teams

Own:

- network intent
- domain-specific workflows
- topology requirements
- domain verification

SRE

Partners on:

- operational standards
- SLOs
- incident management
- capacity

Security

Owns relevant security standards and control requirements.

Cloud/Infrastructure Platform

Owns Kubernetes/runtime infrastructure.

Network Operations

Owns operational response for underlying network incidents.

This avoids:

| "Everyone jointly owns everything."

Shared responsibility must still contain clear decision boundaries.

46. Platform as an Internal Product

Adoption cannot rely only on mandate.

A network engineer will avoid the platform if using it is dramatically harder than running an old script.

Therefore the platform provides:

- Self-service APIs
- Golden workflows
- Documentation
- Examples
- Reusable policies
- Common adapters
- Clear errors
- Sandbox/testing
- Dashboards
- Support model

The internal product proposition becomes:

| Use this platform and you receive identity, audit, resilience, workflow state, secrets, observability and progressive rollout automatically.

That is more powerful than:

| Architecture says you must use it.

47. Measuring Success

Useful outcome metrics include:

Metric	Why it matters
% eligible changes automated	adoption
change lead time	speed
successful change rate	quality
manual intervention	maturity
rollback rate	risk signal
configuration drift	control
MTTR	resilience
platform availability	reliability
cost/change	economics
operator hours saved	productivity
repeat incident rate	learning

Weak vanity metric:

| We executed 17 million API calls.

That does not demonstrate business value.

PART XV — YEARS LATER

48. Technical Debt Appears

After several years the platform contains:

- old adapters
- old API versions
- unsupported device software
- duplicate workflows
- large databases
- outdated authentication integration
- inconsistent schemas
- deprecated libraries
- tightly coupled vendor APIs

This is normal.

The VP creates a capacity allocation model.

For example, not necessarily numerically fixed:

Business capability

+

Reliability

+

Security

+

Modernisation

Modernisation is prioritised according to:

Risk

x

Cost of delay

x

Operational burden

x

Future dependency

A deprecated adapter used by 20 devices is different from a legacy authentication mechanism protecting the entire platform.

49. Multi-Year Evolution

Stage 1 — Orchestration

API
|
Workflow
|
Device automation

Main value:

Standard execution.

Stage 2 — Platform Standardisation

Inventory
+
Adapters
+
Policy
+
Audit
+
Multi-vendor

Main value:

Consistency.

Stage 3 — Desired State

Desired
vs
Observed

Main value:

Drift management.

Stage 4 — Policy Driven

Intent
|
Policy
|
Validated automation

Main value:

Safer self-service.

Stage 5 — Selective Closed Loop

Telemetry
|
Decision
|
Remediation

Main value:

Reduced manual operations.

The organisation does not jump directly to Stage 5 because autonomous infrastructure changes require confidence accumulated through earlier stages.

50. Closed-Loop Automation

A mature use case becomes:

```
Telemetry
|
v
Deviation Detection
|
v
Known Problem?
|
v
Confidence Threshold
|
v
Policy Evaluation
|
v
Blast-Radius Check
|
v
Automated Remediation
|
v
Verification
|
+----- failure -----> stop + human
|
v
Audit
```

Reasonable autonomous candidates could include tightly bounded restoration of approved low-risk configuration such as:

- NTP settings
- logging targets
- known telemetry configuration

- selected metadata drift

Changes likely to retain human approval:

- core BGP policy
- firewall security policy
- WAN topology
- routing redistribution
- data-centre fabric architecture

Autonomy depends on:

Confidence
Blast radius
Reversibility
Observability
Business impact

Every autonomous system also needs:

kill switch
+
audit
+
rate limit
+
confidence threshold
+
human escalation

PART XVI — GOVERNANCE

51. Architecture Governance

MGB establishes architecture principles such as:

Principle 1

No unmanaged privileged credentials.

Principle 2

High-risk changes use progressive rollout.

Principle 3

Desired and observed state remain conceptually distinct.

Principle 4

External APIs are versioned.

Principle 5

Vendor behaviour is isolated behind adapters.

Principle 6

Execution must be observable and auditable.

Architecture Decision Records capture major choices.

Governance also defines:

- technology standards
- API standards
- security requirements
- deprecation policy
- exception process

Good governance says:

| Here are the safe paved roads.

Bad governance says:

| Fill in twelve documents before writing software.

52. Five Important ADRs

ADR 1 — Asynchronous Execution

Context: network operations are long-running.

Options: synchronous requests, scripts, durable workflows.

Decision: durable workflows.

Consequences: greater reliability but greater distributed-system complexity.

ADR 2 — Desired vs Observed State

Context: actual network state can drift.

Options: treat latest config as truth or maintain both states.

Decision: separate them.

Consequences: enables reconciliation but requires clear data ownership.

ADR 3 — Vendor Adapter Layer

Context: heterogeneous hardware.

Options: vendor logic inside workflows or adapter abstraction.

Decision: adapter abstraction.

Consequences: cleaner platform but compatibility testing becomes critical.

Our production incident demonstrated that consequence.

ADR 4 — Progressive Rollout Default

Context: network changes can have large blast radius.

Options: global parallel rollout or progressive deployment.

Decision: progressive rollout.

Consequences: slower completion but dramatically reduced failure impact.

ADR 5 — Central Identity and Secrets

Context: automation has privileged access.

Options: credentials inside automation applications or central security services.

Decision: central enterprise identity/secrets.

Consequences: stronger control but creates critical platform dependencies.

53. VP-Level Trade-Offs

Speed vs Safety

Too many approvals destroy self-service.

Too little control can create outages.

Solution:

Risk-based controls

Low-risk change:

automated approval

High-risk BGP change:

human approval + canary

Standardisation vs Flexibility

One universal workflow model reduces duplication.

But forcing every network domain into an unsuitable abstraction damages usability.

Provide:

Standard core

+

Extension points

Centralisation vs Autonomy

Platform team centrally owns:

identity
execution framework
audit
security

Domain teams own:

network intent
domain logic
verification

Availability vs Cost

Running every component active-active across three continents may improve resilience but massively increase cost and consistency complexity.

The design protects the genuinely critical components more strongly.

Automation vs Humans

Automation performs repetitive execution.

Humans remain involved where judgment and blast radius justify it.

Consistency vs Scale

Strong consistency is valuable for workflow ownership and conflicting device writes.

Eventual consistency may be completely acceptable for analytics dashboards.

One consistency model does not fit every subsystem.

54. Communicating One Decision

Take the decision:

| **All high-risk network changes use progressive rollout by default.**

Engineers

Explain:

- batching algorithm
- task states
- rollback
- thresholds
- topology constraints
- concurrency

Network operators

Explain:

| The system changes a representative small group first and automatically pauses when health signals deteriorate.

Security/Risk

Explain:

| Progressive deployment limits blast radius and generates evidence for each approval and verification gate.

Finance

Explain:

| Changes may take slightly longer, but avoided outages have much higher economic value than a few minutes of execution time.

Senior leadership

Explain:

| We deliberately trade some deployment speed for reduced systemic network risk while still delivering much faster than manual operations.

Same decision.

Different communication.

PART XVII — ROADMAP

55. Multi-Year Technical Roadmap

The roadmap is dependency-driven rather than based on fake precision.

FOUNDATION

Identity

Inventory

Workflow

Audit

Adapters

|

v

CORE AUTOMATION

Standard workflows

Pre/post checks

Progressive rollout

|

v

SCALE

Regional workers

Queue partitioning

Capacity controls

|

v

STANDARDISATION

Common APIs

Golden paths

Multi-vendor models

|

v

SELF-SERVICE

Developer/operator APIs

Reusable workflow catalogue

|

v

OBSERVABILITY

Rich telemetry

Topology-aware health

|

v

DESIRED STATE

Reconciliation

Drift management

|

v

POLICY DRIVEN

Risk-aware execution

Automated guardrails

|

v

SELECTIVE CLOSED LOOP

Low-risk remediation

|

v

LEGACY RETIREMENT

Old scripts

Old Jenkins pipelines
Obsolete adapters

You cannot safely build sophisticated closed-loop remediation before establishing reliable inventory and observed state.

The roadmap reflects engineering dependencies.

56. THE COMPLETE PROJECT STORY

Now compress everything into the single story a VP should mentally retain.

MGB started with a network estate of tens of thousands of devices managed through manual CLI, scripts, Jenkins, vendor tools and cloud automation.

The immediate symptom was slow network delivery.

The deeper problem was fragmented operational control.

Instead of beginning with technology, the engineering leadership team identified six dimensions:

Business
Operations
Software
Security
Risk
Organisation

Discovery showed that inventory itself was inconsistent and that no single source could authoritatively describe the whole network.

The platform therefore adopted explicit data ownership and separated authoritative state from observed state.

A platform vision was established:

| Secure, policy-driven, self-service network automation.

But the first release remained deliberately narrow.

The architecture separated:

Control plane
Execution plane
Network/data plane
Security plane
Observability plane

Long-running device operations were modelled as durable asynchronous workflows.

A queue isolated workflow creation from execution.

Workers provided horizontal scale.

Concurrency controls prevented that scalability from overwhelming the network.

Vendor-specific behaviour was isolated behind adapters.

Where possible, structured interfaces such as NETCONF, RESTCONF, OpenConfig and gNMI were preferred, while legacy CLI remained supported.

The platform introduced:

pre-check
change
post-check

rather than treating configuration delivery as success.

A high-risk routing-policy rollout demonstrated the complete model.

The request passed authentication, authorization, inventory resolution, topology checks, policy validation and approval.

The platform created desired state and captured current state.

A workflow then executed using progressive deployment.

Every action was recorded.

Security relied on central identity, workload identity, short-lived credentials where possible and segmented management access.

The platform ran primarily on Kubernetes, but execution cells were positioned throughout the hybrid infrastructure so automation did not depend on a single connectivity path.

The software itself passed CI/CD, security scanning and progressively controlled release.

The network workflows passed a separate set of validation and rollout gates.

Legacy scripts coexisted with the platform while high-value workflows migrated gradually.

Nine months later the routing incident exposed a weakness.

A vendor adapter incorrectly assumed equivalent behaviour across two OS versions.

The first canaries were not representative of version diversity.

Configuration validation passed.

BGP sessions remained healthy.

But important routes disappeared.

Observability detected route-count anomalies and application-path degradation.

Progressive rollout automatically paused the workflow.

Only fourteen devices were affected rather than more than one thousand.

Rollback restored service.

The investigation did not stop at blaming the adapter developer.

Engineering examined:

architecture
test coverage
capability model
canary design
post-checks
risk policy

The resulting platform became safer.

As the estate grew toward 50,000 devices, APIs and workers scaled horizontally, while databases, discovery and telemetry required more fundamental partitioning and lifecycle strategies.

Costs were managed as TCO rather than infrastructure bills alone.

The platform team built bank-specific orchestration and policy capabilities while consuming enterprise databases, observability, secrets and vendor controllers where appropriate.

Ownership was explicitly divided:

Platform team	-> automation platform
Domain teams	-> network intent
Operations	-> network operations
Security	-> security standards
SRE	-> reliability partnership

The platform increasingly behaved like an internal product.

Teams adopted it because it gave them:

faster delivery
+
audit
+
security
+
rollback
+
observability
+
reusable automation

not simply because an architecture board ordered them to.

Over several years the system evolved:

Automation

- Standardisation
 - Desired State
 - Policy Driven
 - Selective Closed Loop
-

At every stage the VP balanced:

speed vs safety
cost vs resilience
centralisation vs autonomy
standardisation vs flexibility
automation vs human judgment
delivery vs modernisation

That is the project.

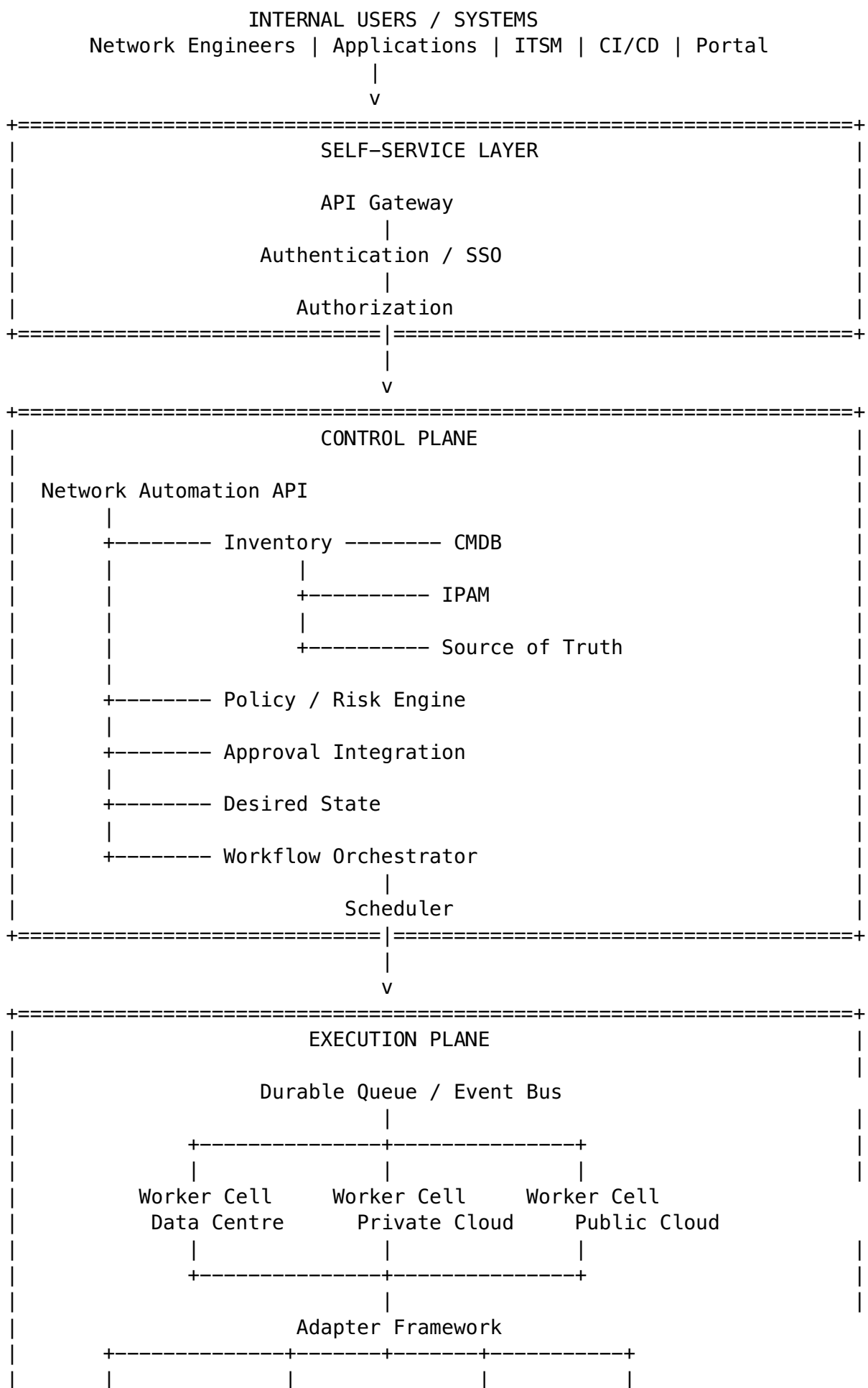
Not:

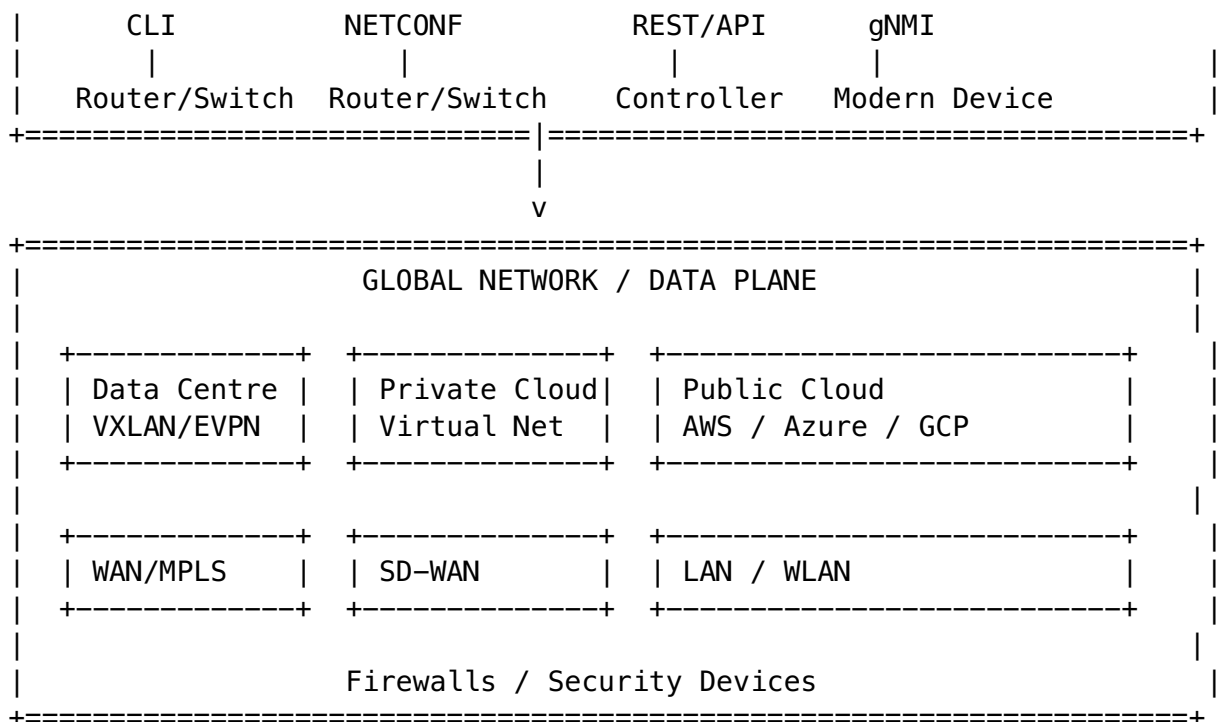
| "We built a network automation application."

But:

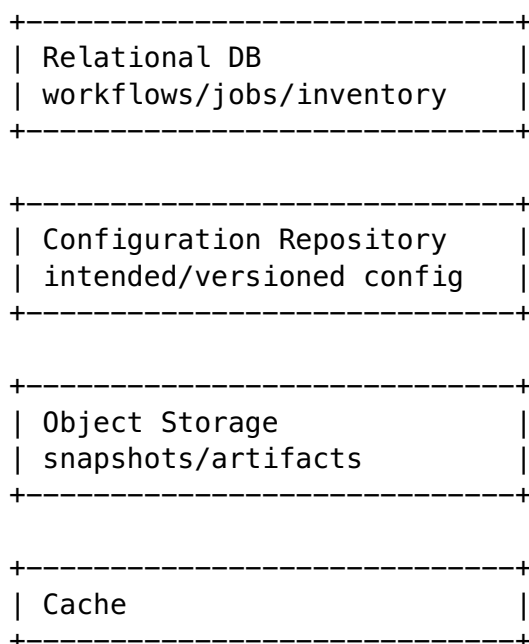
| **We changed how a global institution safely operates its network.**

57. FINAL END-TO-END ARCHITECTURE

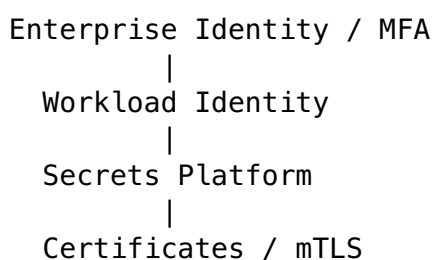




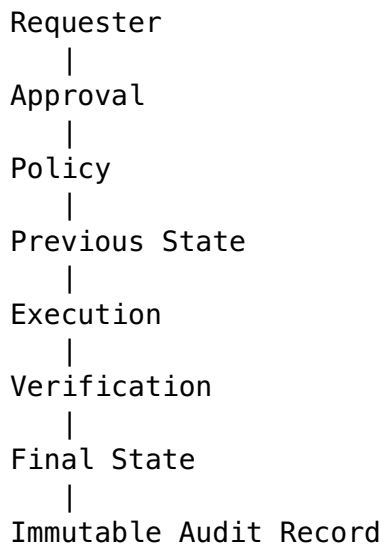
SUPPORTING DATA



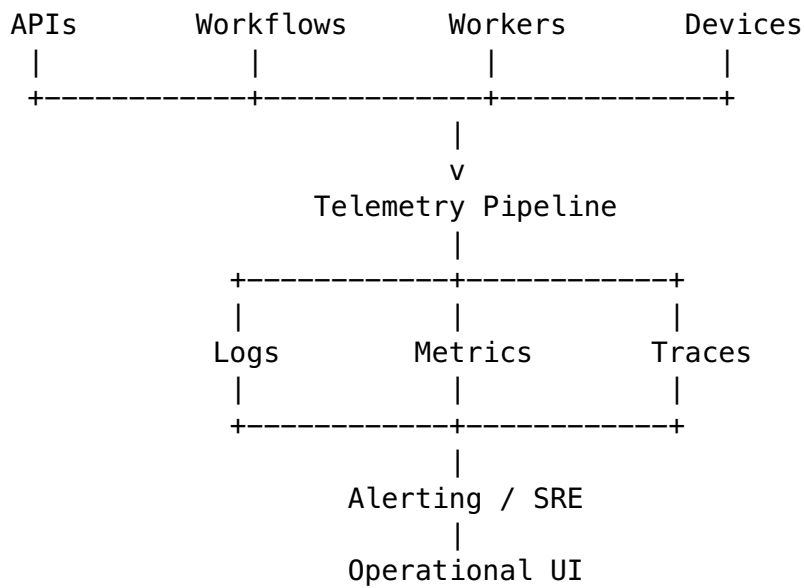
SECURITY PLANE



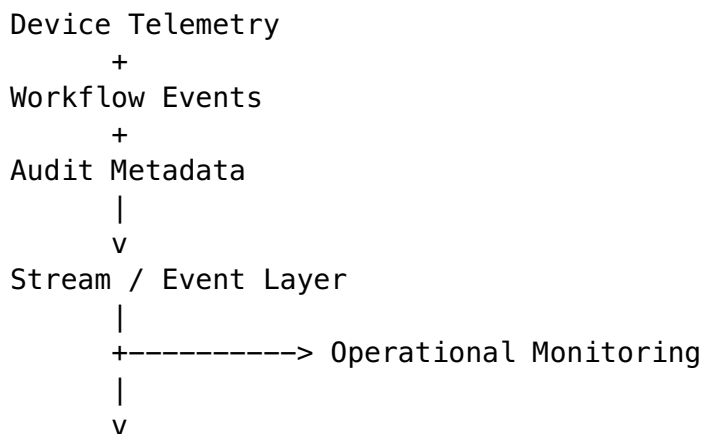
AUDIT / CONTROL PLANE



OBSERVABILITY PLANE



ANALYTICS PLANE



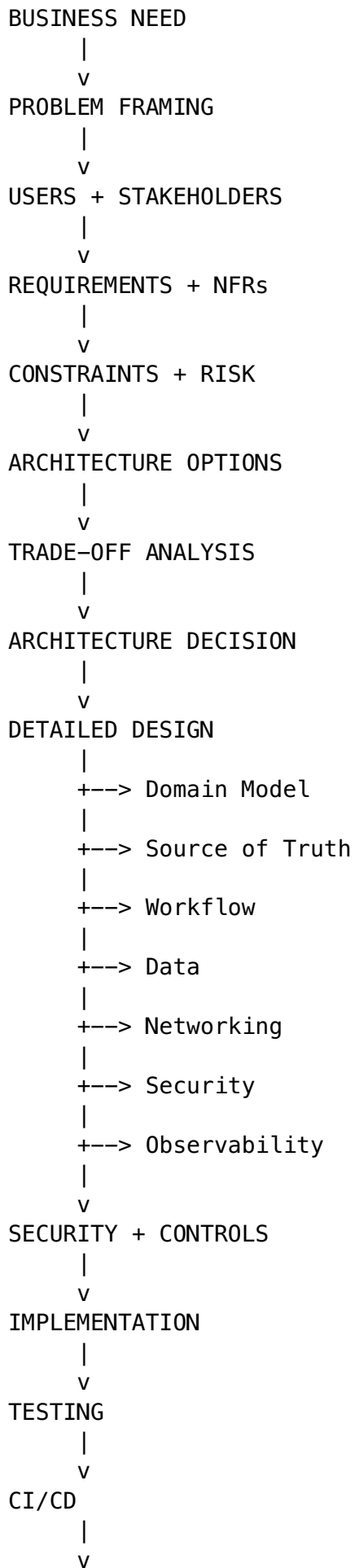
Raw Storage
|
Transformation
|
Analytics Warehouse
|
Dashboards / Risk / Capacity / Trends

CI/CD

Developer
|
Pull Request
|
Review
|
Test
|
Security Scan
|
Build
|
Sign
|
Registry
|
Non-Production
|
Production Canary
|
Controlled Promotion

58. FINAL ENGINEERING LIFECYCLE

The entire Day 7 can finally be represented as:



CONTROLLED PRODUCTION ROLLOUT

|

v

OBSERVABILITY

|

v

OPERATIONS

|

v

INCIDENT

|

v

CONTAINMENT + RECOVERY

|

v

ROOT CAUSE

|

v

ENGINEERING IMPROVEMENT

|

v

SCALE

|

v

COST / CAPACITY / OWNERSHIP

|

v

MODERNISATION

|

v

POLICY-DRIVEN AUTOMATION

|

v

SELECTIVE CLOSED LOOP

|

v

LONG-TERM PLATFORM STRATEGY

The most important Day 7 lesson

At junior or mid-level scope, engineering responsibility can feel like:

Requirement

↓

Code

↓

Test

↓

Done

At Lead/Staff/Principal/VP scope, the responsibility looks much more like:

Why are we doing this?

↓

What problem are we actually solving?

↓

What could go wrong?

↓

What architecture makes failure survivable?

↓

How do teams safely build it?

↓

How do we migrate existing systems?

↓

How do we prove it works?

↓

How do we operate it?

↓

What happens at 3 AM when it fails?

↓

How do we recover?

↓

What do we learn?

↓

How does it scale?

↓

What does it cost?

↓

Who owns it?

↓

How do we prevent today's architecture
from becoming tomorrow's legacy?

That is the major shift in engineering thinking.

A VP-level engineer is therefore not merely responsible for choosing whether the platform should use Kubernetes, queues, APIs, BGP automation, NETCONF or PostgreSQL.

The deeper responsibility is to connect:

Business
+
Architecture
+
Networking
+
Distributed Systems
+
Security
+
Risk
+
Reliability
+
Operations
+
Economics
+
Organisation

and make decisions whose consequences remain manageable **years after the original project has shipped.**

That is what turns this from a network-automation implementation into a **mission-critical enterprise engineering platform.**